

YOLO

■ 상위 작업	📖 세션 논문 리뷰
■ 진행 상태	완료
■ 첨부 파일	YOLO.pdf

[0. Abstract](#)

[1. Introduction](#)

[1. Introduction](#)

[2. Related Work](#)

[3. Method \(제안 방법론\)](#)

[3. Unified Detection](#)

[3.1 Network](#)

[3.2 Training](#)

[3.3. Limitations](#)

[4. Experiments \(실험 및 결과\)](#)

[5. Conclusion](#)

0. Abstract

| 어떤 문제를 해결하려 했는가?

- 기존 객체 탐지 연구는 분류기를 재사용하여 탐지를 수행했다.
- 이에 반해 논문 저자들은 객체 탐지를 regression 문제로 정의하여, bounding box와 클래스 확률을 동시에 예측하는 YOLO를 제안하였다.
- 객체 검출 파이프라인이 단일 신경망으로 구성되어 있어 전체 이미지를 한 번에 처리하는 end-to-end 방식을 따른다.
- 기본 YOLO 모델은 초당 45 프레임을 처리하고, 더 작은 버전인 Fast YOLO는 초당 155프레임을 처리하는 성능을 기록했다.

1. Introduction

| 연구 동기와 문제점은 무엇인가?

1. Introduction

- 사람은 이미지를 순간적으로 보고 객체의 위치와 종류를 빠르게 파악한다. 이러한 능력은 자율주행, 보조 장치, 로봇 시스템 등에 필요한 핵심 기술이다.
- 기존 객체 탐지 시스템은 분류기를 재사용하는 방식으로 대표적으로 DPM과 R-CNN이 있다. 그러나 이들은 복잡하고 느리며, 각 구성 요소를 따로 학습해야 최적화가 어렵다.

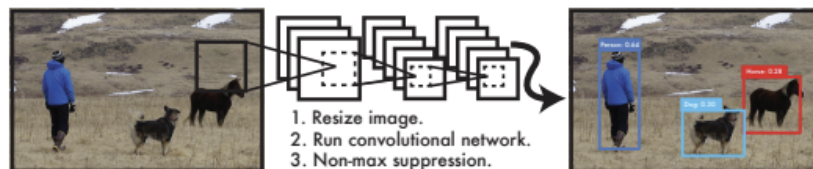


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

- YOLO는 객체 탐지를 단일 회귀 문제로 재정의한다. **이미지를 한 번만 보고 bounding box와 클래스 확률을 동시에 예측한다.**
- YOLO의 이러한 통합된 모델은 기존의 객체 검출 모델에 비해 여러 가지 장점이 있다.
 - YOLO는 단일 신경망 구조이기 때문에 구성이 단순하며, 빠르다.
 - YOLO는 주변 정보까지 학습하며 이미지 전체를 처리하기 때문에 background error가 작다.
 - YOLO는 훈련 단계에서 보지 못한 새로운 이미지에 대해서도 검출 정확도가 높다.
 - 단, YOLO는 SOTA 객체 검출 모델에 비해 정확도(mAP)가 다소 떨어진다.

2. Related Work

관련 연구 동향은 어떠한가?

- 기존의 객체 검출 모델
 - DPM(Deformable parts models)

- 이미지 전체를 거쳐 sliding window 방식으로 훑어 객체 검출을 하는 모델이다.
- 물체를 부분(part) 단위로 분해해 모델링하기 때문에 다양한 형태 변화에 대응할 수 있었다.
- **하지만 sliding window 자체가 계산량이 크고, 실시간 적용에는 부적합했다.**
- R-CNN
 - 이미지 안에서 bounding box를 생성하기 위해 region proposal을 사용하는 모델이다.
 - 제안된 box마다 CNN 특징 추출 → SVM 분류기 → bounding box regression → NMS(중복 제거) 등의 후처리를 거친다.
 - **CNN을 각 proposal마다 독립적으로 적용하기 때문에 연산량이 매우 크고, 학습/추론 속도가 느리다는 단점이 존재했다.**
- 1 stage vs 2 stage
 - 2 stage Detector
 - Classification과 Localization을 순차적으로 진행하는 방법
 - 1 stage Detector
 - Classification과 Localization을 동시에 진행하는 방법
 - Object detection model이 deep learning based로 넘어오게 된 뒤 처음 제안된 1-stage 모델이 YOLO

3. Method (제안 방법론)

┃ 연구 접근법과 모델 구조는 어떻게 되는가?

3. Unified Detection

Main Idea

객체 탐지를 end-to-end 단일 CNN 회귀 문제로 풀어, 이미지 전체에서 bounding box와 클래스 확률을 동시에 예측하자!!

- YOLO는 객체 탐지 과정을 단일 신경망으로 통합한다.
 - 이미지 전체를 입력받아 한 번에 bounding box와 클래스 확률을 동시에 예측한다.
 - 입력 이미지를 S x S grid로 나누고, 각 grid cell은 해당 cell 안에 객체의 중심이 있을 경우 그 객체를 탐지해내야 한다.
 - 각 grid cell은 B개의 bounding box와 confidence score을 예측한다.
 - confidence score

$$\Pr(Object) * IOU_{pred}^{truth}$$

= 객체가 존재할 확률 * 예측 box와 실제 box의 IOU
= 객체가 없으면 $\Pr(Object)=0 \rightarrow$ confidence도 0
= 객체가 있으면 $\Pr(Object)=1 \rightarrow$ confidence는 이상적으로 IOU와 동일

- 각 bounding box는 5개의 예측치로 구성되어 있다.
 - x, y, w, h, confidence
(x, y) = bounding box 중심의 그리드 셀 내의 상대 위치 (0~1)
(w,h) = bounding box의 상대 너비 & 상대 높이 (전체 너비와 높이는 1 $\rightarrow$ (0~1))
confidence = 앞서 구한 confidence score
- 이후 각 그리드 셀은 C개의 conditional class probabilities를 예측한다.
 - conditional class probabilities

$$C(\text{conditional class probabilities}) = \Pr(Class_i|Object)$$

= 그리드 셀 안에 객체가 있다는 조건 하에 그 객체가 어떤 클래스(class)인지에 대한 조건부 확률

→ 그리드 셀에 몇 개의 bounding box가 있는지와는 무관하게 하나의 그리드 셀에는 오직 하나의 클래스(class)에 대한 확률 값만을 구함

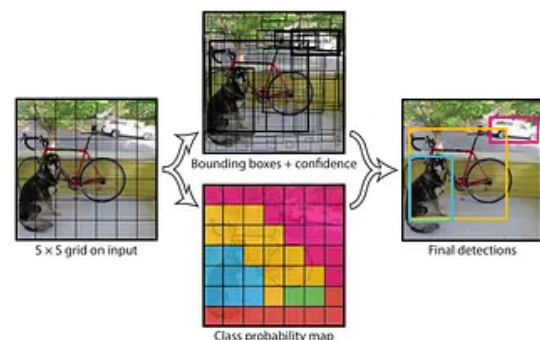
- class specific confidence score

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

- 앞서 구한 **conditional class probability**와 **confidence score**을 곱한 값
- 이 score은 bounding box에 특정 클래스 객체가 나타날 확률($\Pr(\text{Class}_i)$)과 예측된 bounding box가 그 클래스 객체에 얼마나 잘 들어맞는지($\text{IOU}_{\text{pred}}^{\text{truth}}$)를 나타낸다.

▪ YOLO의 Detection 방식

- $S \times S$ 크기의 Grid 생성
- 각 Grid Cell에서 B 개의 Bounding Box 생성
- 각 Bounding Box에서 5 개의 값 생성 ($x, y, w, h, \text{Confidence Score}$)
- C 개의 클래스에 대한 Conditional Class Probabilities 생성
- 최종적으로 예측된 정보는 $S \times S \times (B \times 5 + C)$ tensor로 표현 가능
- PASCAL VOC로 testing할 때
 - $S = 7, B = 2, C = 20$
 - $7 \times 7 \times 30$ tensor 최종 예측 결과



YOLO Detection Summary

3.1 Network

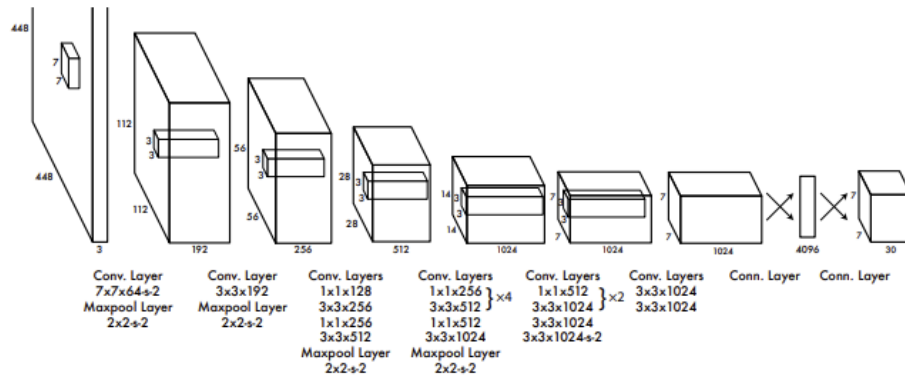


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

- Architecture
 - 24개의 Convolutional Layers를 통해 이미지로부터 특징을 추출한다.
 - Conv layers 뒤의 두 개의 Fully connected layers는 Output Probabilities와 Coordinates를 예측한다.
 - YOLO는 최종으로 $7 \times 7 \times 30$ 크기의 tensor를 Output으로 가진다.
 - Fast YOLO의 경우 24개 대신 9개의 Convolutional layers를 사용했고 각 layer는 더 적은 수의 필터를 가지도록 했다.

3.2 Training

- Pretraining
 - ImageNet의 1000-class Competition Dataset으로 첫 20개의 Convolutional Layers를 Pretrain하였다.
 - ImageNet 2012 Validation set에서 88%의 정확도 달성.
 - 이 모든 훈련과 추론(inference)를 위해 Darknet 프레임워크 사용 (자체개발)
- Detection으로의 전환
 - ImageNet은 분류를 위한 데이터셋이었기 때문에 사전 훈련된 분류 모델을 객체 검출 모델로 변환하는 과정이 필요했다.
 - Fully Connected Layers를 추가했을 때 네트워크의 성능이 향상된다는 선행 연구에 따라 4개의 Convolutional layers와 2개의 Fully Connected Layers 추가함.

- 이때 Fully Connected Layers는 Randomly Initialized 가중치를 가짐
 - Input Resolution을 224x224에서 448x448로 증가시켜 모델의 성능을 올림
 - 신경망의 최종 아웃풋은 클래스 확률(class probabilities)와 bounding box 위치정보(coordinates)
 - bounding box의 너비(w)/높이(h)/중심좌표(x, y)는 모두 0~1로 정규화.
 - 마지막 계층은 선형 활성화 함수를 적용하고, 나머지 모든 계층은 leaky ReLU 적용.
- Loss Function
 - YOLO는 **SSE(sum-squared error)** 기반 loss를 사용하였는데, 단순 SSE에는 문제점이 있다.
 - SSE는 계산과 최적화가 단순하다는 장점이 있지만 실제 목표인 mAP 향상과는 완전히 일치하지는 않음
 - YOLO엔 크게 3가지 loss가 있는데 SSE는 이 세 가지 손실에 동일한 가중치를 두어 단순히 적용하면 localization과 classification의 중요도를 구분하지 못하는 문제가 발생함.
 - Localization Loss: bbox 좌표(x, y, w, h) 예측 정확도
 - Confidence Loss: 객체 존재 여부와 IOU 반영
 - Classification Loss: 클래스 확률 예측
 - 이에 가중치 $\lambda_{coord}=5$, $\lambda_{noobj}=0.5$ 를 적용하였다.
 - 또한 객체가 존재하는 bounding box의 좌표 오차에 더 높은 가중치를 두도록 하였다.
 - 대부분의 그리드 셀에는 실제 객체가 존재하지 않아 많은 셀에서 confidence=0으로 학습됨.
 - 모델이 배경을 맞추는 데 지나치게 치우치고 실제 객체가 있는 셀에 대한 학습 비중이 낮아지는 불균형을 해결하는 방안
 - width와 height에는 제곱근을 적용하였다.
 - 큰 박스와 작은 박스 모두 같은 오차로 계산하는데 이때 작은 객체가 더 불리하게 작용

- 제곱근을 적용하여 작은 박스는 더 민감하게, 큰 박스는 덜 민감하게 반영하도록 함
- 마지막으로, 하나의 객체에 대해 IOU가 가장 큰 predictor가 책임을 가지도록 매칭하였다.
- 하나의 객체를 여러 bounding box predictor가 동시에 예측하면 학습에 혼란이 발생하기 때문

loss function:

$$\begin{aligned}
& \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
& + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
& + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
\end{aligned}$$

최종 Loss Function

3.3. Limitations

1. YOLO는 Strong Spatial Constraints를 부과하는데, 각 Grid Cell은 오직 B개의 Bounding Box를 예측하고 하나의 클래스만 가질 수 있다. Spatial Constraints에 의해 가까이 있는 객체의 수에 대한 예측 제한이 발생한다. 아래의 사진에서 하나의 Cell 안에 여러 마리의 새가 존재하는데 YOLO는 주로 하나의 Cell에서 2개의 객체만 탐지한다는 한계가 있다.



2. YOLO는 특정 데이터로만 학습해서 Bounding box의 가로 세로의 비가 Unusual하거나 New할 경우 탐지 성능이 저하될 수 있다.
3. Loss Function은 Bounding Box의 크기를 고려하지 않는다. Loss Function에서 width와 height에 제곱을 취한 것은 둘의 증가율을 감소시킨 것이지만 본질적으로 작은 Bounding Box가 Error에 더 민감하다는 문제를 해결하진 못한다.

4. Experiments (실험 및 결과)

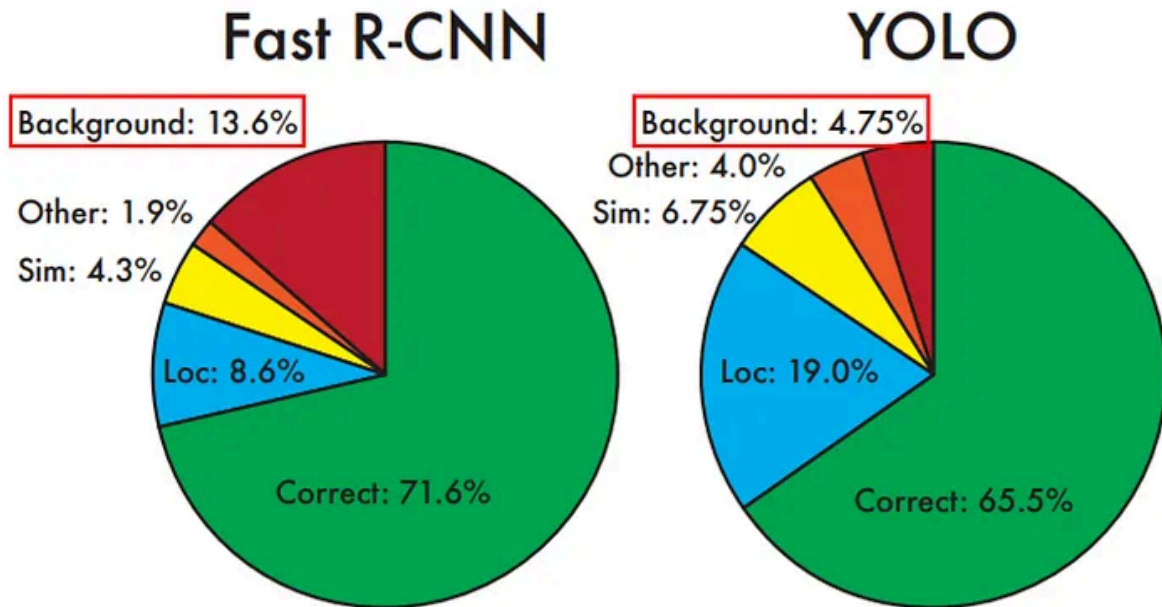
| 실험 구성과 결과는?

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

- Real-Time Detectors
 - Fast YOLO: 155 FPS, 52.7% mAP
→ 가장 빠른 탐지기, 기존 실시간 모델보다 2배 정확
 - YOLO: 45 FPS, 63.4% mAP
→ Fast YOLO보다 정확도 +10 mAP, 여전히 실시간
 - YOLO VGG-16: 더 정확(66.4%)하지만 21 FPS
→ 실시간 불가
- Less Than Real-Time
 - DPM 계열: 속도 ↑ but 정확도 낮음, 실시간 미달 (≤ 30 FPS)
 - R-CNN minus R: 53.5% mAP, 6 FPS → 실시간 불가
 - Fast R-CNN: 70% mAP, 0.5 FPS → 정확도 높지만 너무 느림
 - Faster R-CNN: 62~73% mAP, 7~18 FPS → 정확도 ↑ 그러나 여전히 실시간 불가

⇒ YOLO는 속도(실시간성)+정확도를 동시에 만족시킨 최초의 범용 탐지기



Correct : class가 정확하며 IOU > 0.5 인 경우

Localization : class가 정확하고, $0.1 < \text{IOU} < 0.5$ 인 경우

Similar : class가 유사하고 IOU > 0.1 인 경우

Other : class는 틀렸으나, IOU > 0.1 인 경우

Background : 어떤 Object라도 IOU < 0.1 인 경우

- YOLO
 - localization error 가 상대적으로 크다.
 - localization error는 19.0%로 나머지 error를 모두 합한 15.5% (6.75%+4.0%+4.75%) 보다 크다
- Fast R-CNN
 - YOLO에 비해 localization error가 작다.
 - 반면, background error가 상대적으로 크다.
 - background error는 배경에 아무 물체가 없는데 물체가 있다고 판단하는 false positive error
 - Fast R-CNN은 YOLO에 비해 background error가 3배나 더 크다.

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

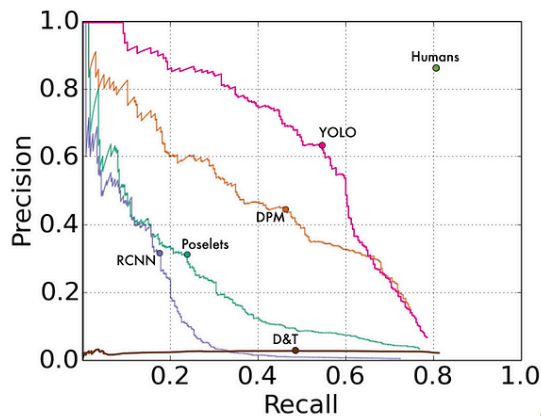
Table 2: Model combination experiments on VOC 2007. We examine the effect of combining various models with the best version of Fast R-CNN. Other versions of Fast R-CNN provide only a small benefit while YOLO provides a significant performance boost.

- Fast R-CNN + YOLO
 - Fast R-CNN과 YOLO를 결합했더니 mAP가 3.2% 올라 75.0%가 되었다.
 - Fast R-CNN과 다른 모델과의 앙상블은 mAP 향상이 0.3%, 0.6%로 미미하다.
 - Fast R-CNN을 사용하는 것보다는 Fast R-CNN과 YOLO를 결합한 모델을 사용하는 것이 더 낫다고 판단할 수 있다.

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR_CNN_MORE_DATA [11]	73.9	85.5	82.9	76.6	57.8	62.7	79.4	77.2	86.6	55.0	79.1	62.2	87.0	83.4	84.7	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet_VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	79.8	87.7	49.6	74.9	52.1	86.0	81.7	83.3	81.8	48.6	73.5	59.4	79.9	65.7
HyperNet_SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	89.4	49.4	75.5	57.0	87.5	80.9	81.0	74.7	41.8	71.5	68.5	82.1	67.2
MR_CNN_S_CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP_ENS_COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	68.8	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH_FGS_STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS_NIN_C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS_NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

Table 3: PASCAL VOC 2012 Leaderboard. YOLO compared with the full comp4 (outside data allowed) public leaderboard as of November 6th, 2015. Mean average precision and per-class average precision are shown for a variety of detection methods. YOLO is the only real-time detector. Fast R-CNN + YOLO is the forth highest scoring method, with a 2.3% boost over Fast R-CNN.

- 파스칼 VOC 2012 데이터 셋
 - YOLO는 57.9%의 mAP를 달성했다.
 - 속도 측면에서는 YOLO가 빠르고, 정확도 측면에서는 Fast R-CNN과 YOLO를 결합한 모델이 가장 좋았다.



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso AP	Picasso Best F_1	People-Art AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 5: Generalization results on Picasso and People-Art datasets.

- YOLO과 다른 객체 검출 모델의 성능 비교
 - 파스칼 VOC 2007에서 학습한 YOLO, R-CNN, DPM 등의 성능을 서로 비교하였다.
 - R-CNN은 VOC 2007에서는 높은 정확도를 보이지만 예술작품에 대해서는 굉장히 낮은 정확도를 보였다.
 - DPM은 예술 작품에 대해서도 정확도가 크게 떨어지지 않는 않지만 정확도도 그리 높지 않았다.
 - YOLO는 VOC 2007에서도 가장 높은 정확도를 보였고, 예술 작품에 대해서도 정확도가 크게 떨어지지 않았다.

5. Conclusion

결론 및 주요 요약은?

- YOLO는 객체 검출을 단일 회귀 문제로 정의하여 end-to-end 방식으로 학습할 수 있게 했다.

- 이로써 실시간 속도(45 FPS, Fast YOLO는 155 FPS)와 준수한 정확도를 동시에 달성했으며, 새로운 도메인에도 잘 일반화되는 장점을 보였다.
- 다만 작은 객체나 근접한 객체에서는 성능이 떨어지고 localization error가 두드러지는 한계가 있었다.
- 그럼에도 YOLO는 정확도와 속도의 균형을 잡은 최초의 실시간 객체 검출 모델로서 큰 의의를 가진다.